ChatGPT

Collation and String Comparison in Amazon Redshift

Amazon Redshift does **not** support locale-specific or user-defined collations. Instead, it always compares and sorts text using a simple binary UTF-8 ordering [1]. In practice, that means Redshift treats strings as sequences of bytes: `'A'` (0x41) sorts before `'a'` (0x61), and accented characters like `'á'` (0xC3A1) sort after all basic ASCII letters. There are no rules for language-specific ordering – Redshift ignores linguistic conventions and accents by default [1]. In other words, without intervention, **string comparisons and ORDER BY operate on raw codepoint values**.

By default, Redshift uses a **case-sensitive** collation. That is, `'A' <> 'a'` and the default `LIKE` is case-sensitive [2]. (Oracle, Teradata, etc. users often ask for case-insensitive behavior, but out-of-the-box Redshift is case-sensitive unless you specify otherwise.) Under the hood, the database sorts and compares `CHAR` / `VARCHAR` columns by binary values. For example, in a simple `ORDER BY name`, all uppercase names will typically appear before any lowercase names, because their UTF-8 bytes are smaller. Likewise, `WHERE name = 'John'` will match only rows where `name` exactly equals `'John'` in case and spelling. In summary: **no locale, no accent rules** – just byte-by-byte comparison [1].

Case-Insensitive Collation Support

Starting in mid-2021, Redshift introduced limited support for case-insensitive collation. You can now specify collations at the database, table, column, or expression level [3] [4]. For example, when creating a new database you can say:

```
CREATE DATABASE mydb WITH COLLATE CASE_INSENSITIVE;
```

By default (if you omit `COLLATE`), the database uses **case-sensitive** collation [5]. If you create a **case-insensitive** database, *all* new `CHAR` / `VARCHAR` columns in it will be case-insensitive by default [6] [7]. You can also override the collation on individual columns in a `CREATE TABLE` statement:

```
CREATE TABLE users (
  firstname VARCHAR(50) COLLATE case_insensitive,
  code      CHAR(10)    COLLATE case_sensitive,
  addr      VARCHAR(100)  -- inherits database collation
);
```

Finally, at **query time** you can override collations using the `COLLATE()` function. For example:

```
SELECT *
FROM users
WHERE COLLATE(firstname,'case_insensitive') = 'alice';
```

This treats the comparison as case-insensitive for that expression. Redshift only supports `case_sensitive` and `case_insensitive` as options [4].

When a column (or database) is marked case-insensitive, Redshift ignores case differences in **comparisons, GROUP BY, ORDER BY, DISTINCT, LIKE/ILIKE, window partitions, MIN/MAX, LISTAGG**, etc. [7]. For instance, if `firstname` is case-insensitive, then `WHERE firstname='John'` will match both `'John'` and `'JOHN'`. The data preserves its original case, but all string operations treat uppercase and lowercase as equivalent [7]. Pattern matches like `ILIKE` also follow this rule. (Redshift's `ILIKE` is already case-insensitive for ASCII characters [2]; with a case-insensitive collation, even multibyte letters follow suit.)

Workarounds for Case-Insensitive and Accent-Insensitive Comparison

Without collation support (older versions) or when working with mixed collations, the usual workaround is to normalize the case in your queries. A common approach is to convert both sides of a comparison to upper (or lower) case. For example:

```
SELECT *
FROM users
WHERE UPPER(firstname) = UPPER('alice');
```

This ensures `'Alice'`, `'ALICE'`, and `'alice'` all match. Alternatively, use `ILIKE` or `LIKE LOWER(...)`: by default, `ILIKE` is a case-insensitive pattern match **for single-byte (ASCII) characters** [2]. For multibyte or accented characters, Redshift recommends using `LOWER()` and `LIKE`, e.g.:

```
SELECT *
FROM cities
WHERE LOWER(city_name) LIKE LOWER('%münch%');
```

Because Redshift's `ILIKE` only ignores case for ASCII, accents and other UTF-8 characters require explicit normalization [2].

Redshift has **no built-in accent-insensitive collation**. If you need to ignore accents (diacritics), you must manually normalize or strip them. A practical method is to use `TRANSLATE` or `REGEXP_REPLACE` to replace accented characters with their plain equivalents. For example, the Redshift `translate` function can map many characters at once. One user showed how:

```
SELECT TRANSLATE(
  'São Paulo',
  'áàãâäéèêëíìîïóòõôöúùûüçÁÀÃÄÉÈÊËÍÌÎÏÓÒÕÖÔÚÙÛÜÇ',
  'aaaaaeeeeiiiioooooouuuucAAAAAEEEEIIIIOOOOOUUUUC'
);
-- Returns: 'Sao Paulo'
```

This replaces *all* common accented letters with unaccented ones in one call [8]. You can wrap it in `LOWER()` if you also want case normalization. In general, you may need to apply a chain of replaces or a UDF if your data has many languages.

Sorting, Filtering, and Comparison Examples

Below are some examples of how Redshift handles text sorting and filtering:

- **Case-sensitive column (default)**:

```
CREATE TEMP TABLE T (s VARCHAR(10));
INSERT INTO T VALUES ('Apple'),('apple'),('Banana'),('banana');
SELECT * FROM T ORDER BY s;
```

Result (case-sensitive sort): `('Apple','Banana','apple','banana')`. Uppercase come first by binary order.

```
SELECT * FROM T WHERE s = 'Apple';
```

Matches only `'Apple'`, not `'apple'`.

- **Case-insensitive column**:
If we declare `s VARCHAR(10) COLLATE case_insensitive`, then the above `WHERE s = 'Apple'` would return both `'Apple'` and `'apple'`, and `ORDER BY s` would treat `'Apple'` and `'apple'` as equal for sort order [7]. (Redshift keeps their original case but sorts them together.)

```
SELECT *
FROM T
WHERE COLLATE(s,'case_insensitive') = 'apple';
```

This would match both `'Apple'` *and* `'apple'` [9] [10]. In fact, the AWS docs example shows exactly this effect: a case-sensitive column finds only one match, while forcing case-insensitive with `COLLATE` returns both [9] [10].

- **Using UPPER/LOWER**:

```
SELECT *
FROM users
WHERE UPPER(lastname) = UPPER('o\'reilly');
```

This treats `O'Reilly`, `o'REILLY`, etc. as equal in comparison, effectively a case-insensitive filter (though it does extra work at query time).

- **Pattern matching**:

```
SELECT * FROM products
WHERE name ILIKE '%widget%';
```

This matches `'Widget'`, `'widget'`, etc. But be careful: `ILIKE` won't ignore accents, so `'á'` vs `'a'` differences will still matter [2]. For multi-byte case-insensitive search, use `LOWER(...) LIKE LOWER(...)` as shown above.

- **Sorting by case-insensitive logic**:
If you have a case-insensitive column (or override), then `ORDER BY` ignores case. For example, with case-insensitive collation, `ORDER BY name` would treat `Alice` and `alice` as equal for ordering, so they may appear grouped (and sorted by their lowercase values). Unfortunately AWS examples don't explicitly show this, but the same principle applies to ORDER BY and GROUP BY when collation is case-insensitive [7].

Importing Data and Multilingual Considerations

When importing external data (AWS Glue, Spectrum external tables on S3, etc.), keep these points in mind:

- **Encoding**: Redshift expects UTF-8 text. By default the `COPY` command assumes UTF-8 (as do external tables) [11]. You can use `ENCODING 'UTF8'` (default) or specify other encodings like `ISO88591` if needed [11]. In any case, *all load data must use the declared encoding*, or `COPY` will fail [12]. For example, if loading a CSV with French accents, ensure the file is actually UTF-8 or use `ACCEPTINVCHARS` to replace bad bytes [13]. If Redshift sees invalid UTF-8 during `COPY`, it raises an error (or, with `ACCEPTINVCHARS`, replaces invalid chars with a placeholder) [13] [12].

- **Glues/Spectrum**: If you query data in S3 via Redshift Spectrum (using Glue catalog), Redshift still applies **its own** collation rules. Spectrum does not introduce new collations; external queries inherit the **current database's collation** [14]. So if your Redshift DB is case-insensitive, a Spectrum `SELECT` will also treat those VARCHAR columns case-insensitively [14]. Data in external tables is read as bytes and then subject to Redshift's comparison rules.

- **Multilingual data**: Redshift treats every character by its codepoint, so data from various languages will be ordered by Unicode values. For example, characters like `ñ` or `ç` have higher codepoints

than basic letters and will sort later. There is no built-in locale or accent sensitivity. To manage this, you may need to *normalize* strings in ETL or use language-specific processing before loading. Common tactics include: converting text to a single case (e.g. all uppercase), removing or translating diacritics (using scripts or UDFs), or storing an additional "sort key" column with normalized text.

- **Other sources**: When migrating from systems with collations (Oracle, SQL Server, etc.), remember that Redshift **will not** honor those collations. All comparisons fall back to Redshift's binary rules. You may need to adjust queries or data (e.g. uppercasing indexes) during migration.

Limitations and Pitfalls

- **Limited collation options**: Redshift only supports two collations: `CASE_SENSITIVE` and `CASE_INSENSITIVE` [4]. You *cannot* define custom collations or choose a language locale (no "en_US" or accent rules). If you need accent-insensitivity or other locale behaviors, you must handle them manually (see above).

- **Changing collation**: The database-level collation can only be set at creation (or with `ALTER DATABASE` when empty) [15] [16]. You cannot change a non-empty database's collation. Likewise, changing a table or column's collation requires dropping/recreating it. Plan your collation strategy before loading data.

- **Mixed collations**: Queries that mix case-sensitive and case-insensitive columns are not supported. Redshift will throw an error if you compare or join a CI column to a CS column [17]. For example, `SELECT * FROM t WHERE ci_col = cs_col;` is disallowed [17]. You must keep collations uniform in any given operation, or explicitly override one side with `COLLATE()`.

- **Leader node queries**: Case-insensitive collation is **not** supported in leader-only queries (such as certain DDL or metadata queries) [18]. Typically this does not affect normal SELECT queries, but be aware of it when using system functions or pages.

- **System tables**: Redshift's system catalogs and pg_catalog views are always case-sensitive. For example, the user name in `pg_user` is case-sensitive no matter what your DB collation is [19].

- **Performance**: Using `LOWER()` or `UPPER()` on every comparison can slow queries, since it prevents index/scan optimization. Collations incur minimal overhead, but using functions on large text columns may not. Test your workload for any slowdown.

Best Practices for Consistent Ordering

To ensure predictable string behavior across regions and languages:

- **Use one collation consistently**: If you have multiple Redshift clusters or databases (even in different regions), choose case-sensitive or case-insensitive and stick with it. Cross-database queries **cannot** run if collations differ [20]. For consistent results, set the same collation on each.

- **Normalize data on load**: When dealing with multilingual data, it often helps to standardize text as you load it (e.g. `COPY` with `ENCODING 'UTF8'`, then `UPDATE` to `LOWER()` every name). You might store a "normalized" version of the text for comparisons.

- **Use explicit sorting keys**: If sort order matters (e.g. in reports), consider adding an explicit sort key column that's been pre-processed for your needs. For example, a column that holds all-lowercase ASCII text for each name ensures a reproducible order.

- **Document your collation strategy**: Because Redshift's behavior is simpler than other databases, it's important for teams to know how you're handling case and accents. Document whether your warehouse is using `CASE_INSENSITIVE` or relies on uppercasing, and train users accordingly.

- **Test with sample data**: Before loading production data, try queries on sample multilingual text to see how Redshift sorts and filters. For instance, load words with accents and run `ORDER BY` to see their order, then adjust your approach if needed.

By understanding Redshift's binary collation model and using functions or the new `COLLATE` feature, you can control string comparison behavior. Remember that **Redshift will never apply any locale-specific rules**; it always falls back to Unicode codepoint order unless you explicitly transform the data or query. For example, using `COLLATE` or converting text with `LOWER`/`UPPER` achieves case-insensitive comparisons [4] [2], and using `TRANSLATE` or regex can handle accents [8]. Keeping these strategies in mind will help avoid surprises when sorting or matching text across diverse datasets.

Sources: Amazon Redshift documentation and AWS blogs describe how Redshift handles collation and case sensitivity [3] [7] [1]. Official docs on `COLLATE`, `COPY`, and pattern matching provide syntax and examples [4] [2] [11] [13]. Developers have documented practical workarounds (e.g. using `translate` to remove accents [8]) which are also included here.

---

1 Collation sequences - Amazon Redshift
https://docs.aws.amazon.com/redshift/latest/dg/c_collation_sequences.html

2 LIKE - Amazon Redshift
https://docs.aws.amazon.com/redshift/latest/dg/r_patternmatching_condition_like.html

3 5 6 15 16 Case-insensitive collation support for string processing in Amazon Redshift | AWS Big Data Blog
https://aws.amazon.com/blogs/big-data/case-insensitive-collation-support-for-string-processing-in-amazon-redshift/

4 9 10 COLLATE function - Amazon Redshift
https://docs.aws.amazon.com/redshift/latest/dg/r_COLLATE.html

7 14 17 18 19 20 CREATE DATABASE - Amazon Redshift
https://docs.aws.amazon.com/redshift/latest/dg/r_CREATE_DATABASE.html

8 sql - What's the best way to 'normalize' a string in Redshift? - Stack Overflow
https://stackoverflow.com/questions/65206222/whats-the-best-way-to-normalize-a-string-in-redshift

11 12 Data conversion parameters - Amazon Redshift
https://docs.aws.amazon.com/redshift/latest/dg/copy-parameters-data-conversion.html

13 Multibyte character load errors - Amazon Redshift
https://docs.aws.amazon.com/redshift/latest/dg/multi-byte-character-load-errors.html